

FULL STACK WEB DEVELOPMENT
ASSIGNMENT – 2
MONGODB – STUDENT INFORMATION MANAGEMENT SYSTEM

Submitted By

Ch Venu Gopal Reddy

Roll Number: A24126552078

Subject

Full Stack Web Development

Academic Assignment

Objective

Develop a simple system to store, retrieve, update, and manage student information using MongoDB.

MongoDB Installation and Setup

Before performing the assignment, MongoDB Server and MongoDB Shell (mongosh) should be installed. The following checks can be performed in PowerShell or the VS Code terminal.

1. Check MongoDB Shell

```
mongosh --version
```

The installed MongoDB Shell version should be displayed.

2. Check MongoDB Server Service

```
Get-Service MongoDB
```

The MongoDB service should show Running when the server is active.

3. Open MongoDB Shell

```
mongosh
```

After a successful connection, the mongosh prompt is displayed.

```
PS C:\Users\venug> mongosh
Current Mongosh Log ID: 6ac0e0587d8e0c136eb05396
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB:  9.0.2
Using Mongosh:  2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, pseudo-anonymous usage data is collected from interactive and agentic sessions only, and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-10-03T16:25:08.559+05:30: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> show dbs
```

Student Task

Create a MongoDB database named:

```
collegeDB
```

Create a collection named:

```
students
```

Each student document should contain:

```
{
  rollNo: "23CM001",
  name: "Ravi Kumar",
```

```
branch: "CSE-AIML",
year: 3,
marks: 85,
email: "ravi@example.com"
}
```

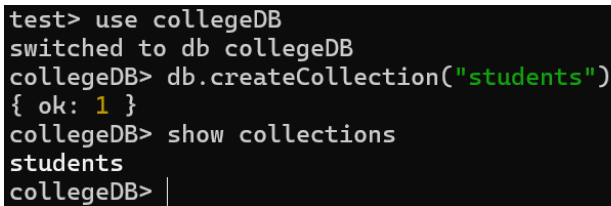
1. Create Database and Collection

Select collegeDB and explicitly create the students collection.

MongoDB Commands:

```
use collegeDB
db.createCollection("students")
show collections
```

Expected Output / Screenshot:



```
test> use collegeDB
switched to db collegeDB
collegeDB> db.createCollection("students")
{ ok: 1 }
collegeDB> show collections
students
collegeDB> |
```

2. Insert Student Records

Insert five student records so that all required operations can be demonstrated.

MongoDB Commands:

```
db.students.insertMany([
  { rollNo:"23CM001", name:"Ravi Kumar", branch:"CSE-AIML", year:3, marks:85,
    email:"ravi@example.com" },
  { rollNo:"23CM002", name:"Anita Sharma", branch:"CSE", year:2, marks:92,
    email:"anita@example.com" },
  { rollNo:"23CM003", name:"Kiran Reddy", branch:"ECE", year:3, marks:68,
    email:"kiran@example.com" },
  { rollNo:"23CM004", name:"Priya Singh", branch:"CSE-AIML", year:4, marks:45,
    email:"priya@example.com" },
  { rollNo:"23CM005", name:"Arjun Rao", branch:"IT", year:2, marks:78,
    email:"arjun@example.com" }
])
```

Expected Output / Screenshot:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac0e1057d8e0c136eb05397'),
    '1': ObjectId('6ac0e1057d8e0c136eb05398'),
    '2': ObjectId('6ac0e1057d8e0c136eb05399'),
    '3': ObjectId('6ac0e1057d8e0c136eb0539a'),
    '4': ObjectId('6ac0e1057d8e0c136eb0539b')
  }
}
```

3. Display All Students

Use find() to display all student documents.

MongoDB Commands:

```
db.students.find()
```

Expected Output / Screenshot:

```
collegeDB> db.students.find().pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AI ML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05398'),
    rollNo: '23CM002',
    name: 'Anita Sharma',
    branch: 'CSE',
    year: 2,
    marks: 92,
    email: 'anita@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05399'),
    rollNo: '23CM003',
    name: 'Kiran Reddy',
    branch: 'ECE',
    year: 3,
    marks: 68,
    email: 'kiran@example.com'
  },
]
```

```
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb0539a'),
    rollNo: '23CM004',
    name: 'Priya Singh',
    branch: 'CSE-AIML',
    year: 4,
    marks: 45,
    email: 'priya@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb0539b'),
    rollNo: '23CM005',
    name: 'Arjun Rao',
    branch: 'IT',
    year: 2,
    marks: 78,
    email: 'arjun@example.com'
  }
]
```

4. Display Students Belonging to a Particular Branch

Display students belonging to the CSE-AIML branch.

MongoDB Commands:

```
db.students.find({ branch: "CSE-AIML" })
```

Expected Output / Screenshot:

```
collegeDB> db.students.find({ branch: "CSE-AIML" }).pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb0539a'),
    rollNo: '23CM004',
    name: 'Priya Singh',
    branch: 'CSE-AIML',
    year: 4,
    marks: 45,
    email: 'priya@example.com'
  }
]
```

5. Display Students Who Scored More Than 75 Marks

Use \$gt to find marks greater than 75.

MongoDB Commands:

```
db.students.find({ marks: { $gt: 75 } })
```

Expected Output / Screenshot:

```
collegeDB> db.students.find({ marks: { $gt: 75 } }).pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05398'),
    rollNo: '23CM002',
    name: 'Anita Sharma',
    branch: 'CSE',
    year: 2,
    marks: 92,
    email: 'anita@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb0539b'),
    rollNo: '23CM005',
    name: 'Arjun Rao',
    branch: 'IT',
    year: 2,
    marks: 78,
    email: 'arjun@example.com'
  }
]
```

6. Search for a Student Using rollNo

Find one student using the rollNo field.

MongoDB Commands:

```
db.students.findOne({ rollNo: "23CM001" })
```

Expected Output / Screenshot:

```
collegeDB> db.students.findOne({ rollNo: "23CM001" })
{
  _id: ObjectId('6ac0e1057d8e0c136eb05397'),
  rollNo: '23CM001',
  name: 'Ravi Kumar',
  branch: 'CSE-AIML',
  year: 3,
  marks: 85,
  email: 'ravi@example.com'
}
```

7. Search Students Based on a Specified Condition

Search using conditions on year and marks.

MongoDB Commands:

```
db.students.find({ year: 3 })
db.students.find({ marks: { $gte: 80 } })
```

Expected Output / Screenshot:

```
collegeDB> db.students.find({ year: 3 }).pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AI ML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05399'),
    rollNo: '23CM003',
    name: 'Kiran Reddy',
    branch: 'ECE',
    year: 3,
    marks: 68,
    email: 'kiran@example.com'
  }
]
```

```
collegeDB> db.students.find({ marks: { $gte: 80 } }).pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AI ML',
    year: 3,
    marks: 85,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05398'),
    rollNo: '23CM002',
    name: 'Anita Sharma',
    branch: 'CSE',
    year: 2,
    marks: 92,
    email: 'anita@example.com'
  }
]
```

8. Update the Marks of a Particular Student

Update Ravi Kumar's marks from 85 to 90 and verify the record.

MongoDB Commands:

```
db.students.updateOne(
  { rollNo: "23CM001" },
  { $set: { marks: 90 } }
)
db.students.findOne({ rollNo: "23CM001" })
```

Expected Output / Screenshot:

```
collegeDB> db.students.updateOne(
|   { rollNo: "23CM001" },
|   { $set: { marks: 90 } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
collegeDB> db.students.findOne({ rollNo: "23CM001" })
{
  _id: ObjectId('6ac0e1057d8e0c136eb05397'),
  rollNo: '23CM001',
  name: 'Ravi Kumar',
  branch: 'CSE-AIIML',
  year: 3,
  marks: 90,
  email: 'ravi@example.com'
}
```

9. Update Another Field – Email

Update Kiran Reddy's email and verify the record.

MongoDB Commands:

```
db.students.updateOne(
  { rollNo: "23CM003" },
  { $set: { email: "kiran.reddy@example.com" } }
)
db.students.findOne({ rollNo: "23CM003" })
```

Expected Output / Screenshot:

```
collegeDB> db.students.updateOne(
|   { rollNo: "23CM003" },
|   { $set: { email: "kiran.reddy@example.com" } }
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
collegeDB> db.students.findOne({ rollNo: "23CM003" })
{
  _id: ObjectId('6ac0e1057d8e0c136eb05399'),
  rollNo: '23CM003',
  name: 'Kiran Reddy',
  branch: 'ECE',
  year: 3,
  marks: 68,
  email: 'kiran.reddy@example.com'
}
```

10. Delete a Student Record Using rollNo

Delete the record with rollNo 23CM005 and verify the remaining records.

MongoDB Commands:

```
db.students.deleteOne({ rollNo: "23CM005" })
db.students.find()
```

Expected Output / Screenshot:

```
collegeDB> db.students.deleteOne({ rollNo: "23CM005" })
{ acknowledged: true, deletedCount: 1 }
```

```
collegeDB> db.students.find().pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05398'),
    rollNo: '23CM002',
    name: 'Anita Sharma',
    branch: 'CSE',
    year: 2,
    marks: 92,
    email: 'anita@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05399'),
    rollNo: '23CM003',
    name: 'Kiran Reddy',
    branch: 'ECE',
    year: 3,
    marks: 68,
    email: 'kiran.reddy@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb0539a'),
    rollNo: '23CM004',
    name: 'Priya Singh',
    branch: 'CSE-AIML',
    year: 4,
    marks: 45,
    email: 'priya@example.com'
  }
]
```

11. Display Students in Descending Order of Marks

Use `sort({marks:-1})` to display highest marks first.

MongoDB Commands:

```
db.students.find().sort({ marks: -1 })
```

Expected Output / Screenshot:

```
collegeDB> db.students.find().sort({ marks: -1 }).pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05398'),
    rollNo: '23CM002',
    name: 'Anita Sharma',
    branch: 'CSE',
    year: 2,
    marks: 92,
    email: 'anita@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05399'),
    rollNo: '23CM003',
    name: 'Kiran Reddy',
    branch: 'ECE',
    year: 3,
    marks: 68,
    email: 'kiran.reddy@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb0539a'),
    rollNo: '23CM004',
    name: 'Priya Singh',
    branch: 'CSE-AIIML',
    year: 4,
    marks: 45,
    email: 'priya@example.com'
  }
]
```

12. Create an Index on rollNo

Create an index on rollNo and display the collection indexes.

MongoDB Commands:

```
db.students.createIndex({ rollNo: 1 })
db.students.getIndexes()
```

Expected Output / Screenshot:

```
collegeDB> db.students.createIndex({ rollNo: 1 })
rollNo_1
collegeDB> db.students.getIndexes()
[
  {
    v: 2,
    key: { _id: 1 },
    name: '_id_',
    collation: { locale: 'simple' }
  },
  {
    v: 2,
    key: { rollNo: 1 },
    name: 'rollNo_1',
    collation: { locale: 'simple' }
  }
]
```

13. Demonstrate Why Indexing Is Useful

Use explain() to inspect the execution plan for a rollNo search. An index scan such as IXSCAN can demonstrate index usage.

MongoDB Commands:

```
db.students.find({ rollNo: "23CM001" }).explain("executionStats")
```

Expected Output / Screenshot:

```

executionTimeMicros: 70676,
totalKeysExamined: 1,
totalDocsExamined: 1,
executionStages: {
  isCached: false,
  stage: 'FETCH',
  nReturned: 1,
  executionTimeMillisEstimate: 46,
  works: 2,
  advanced: 1,
  needTime: 0,
  needYield: 0,
  saveState: 1,
  restoreState: 1,
  isEOF: 1,
  nss: 'collegeDB.students',
  docsExamined: 1,
  alreadyHasObj: 0,
  inputStage: {
    stage: 'IXSCAN',
    nReturned: 1,
    executionTimeMillisEstimate: 46,
    works: 2,
    advanced: 1,
    needTime: 0,
    needYield: 0,
    saveState: 1,
    restoreState: 1,
    isEOF: 1,
    nss: 'collegeDB.students',
    keyPattern: { rollNo: 1 },
    indexName: 'rollNo_1',
    isMultiKey: false,
    multiKeyPaths: { rollNo: [ ] },
    isUnique: false,
    isSparse: false,
    isPartial: false,
    indexVersion: 2,
    direction: 'forward',
    indexBounds: { rollNo: [ ["23CM001", "23CM001"] ] },
    keysExamined: 1,
    seeks: 1,
    dupsTested: 0,
    dupsDropped: 0,
    peakTrackedMemBytes: 0
  }
}

```

Real-Time Extension ★

Perform additional queries on the student records.

14. Find Students Scoring Above 80

Find students with marks greater than 80.

MongoDB Commands:

```
db.students.find({ marks: { $gt: 80 } })
```

Expected Output / Screenshot:

```

collegeDB> db.students.find({ marks: { $gt: 80 } }).pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AI ML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05398'),
    rollNo: '23CM002',
    name: 'Anita Sharma',
    branch: 'CSE',
    year: 2,
    marks: 92,
    email: 'anita@example.com'
  }
]

```

15. Find Students Scoring Below 50

Find students with marks less than 50.

MongoDB Commands:

```
db.students.find({ marks: { $lt: 50 } })
```

Expected Output / Screenshot:

```
collegeDB> db.students.find({ marks: { $lt: 50 } }).pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb0539a'),
    rollNo: '23CM004',
    name: 'Priya Singh',
    branch: 'CSE-AIML',
    year: 4,
    marks: 45,
    email: 'priya@example.com'
  }
]
```

16. Find the Highest-Scoring Student

Sort by descending marks and limit the result to one student.

MongoDB Commands:

```
db.students.find().sort({ marks: -1 }).limit(1)
```

Expected Output / Screenshot:

```
collegeDB> db.students.find().sort({ marks: -1 }).limit(1)
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05398'),
    rollNo: '23CM002',
    name: 'Anita Sharma',
    branch: 'CSE',
    year: 2,
    marks: 92,
    email: 'anita@example.com'
  }
]
```

17. Find Students Belonging to a Particular Branch

Find students from CSE-AIML.

MongoDB Commands:

```
db.students.find({ branch: "CSE-AIML" })
```

Expected Output / Screenshot:

```
collegeDB> db.students.find({ branch: "CSE-AIML" }).pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb0539a'),
    rollNo: '23CM004',
    name: 'Priya Singh',
    branch: 'CSE-AIML',
    year: 4,
    marks: 45,
    email: 'priya@example.com'
  }
]
```

18. Display Students Sorted According to Marks

Display students in ascending order of marks.

MongoDB Commands:

```
db.students.find().sort({ marks: 1 })
```

Expected Output / Screenshot:

```
collegeDB> db.students.find().sort({ marks: 1 }).pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb0539a'),
    rollNo: '23CM004',
    name: 'Priya Singh',
    branch: 'CSE-AIML',
    year: 4,
    marks: 45,
    email: 'priya@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05399'),
    rollNo: '23CM003',
    name: 'Kiran Reddy',
    branch: 'ECE',
    year: 3,
    marks: 68,
    email: 'kiran.reddy@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIML',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05398'),
    rollNo: '23CM002',
    name: 'Anita Sharma',
    branch: 'CSE',
    year: 2,
    marks: 92,
    email: 'anita@example.com'
  }
]
```

19. Final Display of Student Records

Display the final student records after the update and delete operations.

MongoDB Commands:

```
db.students.find()
```

Expected Output / Screenshot:



```
collegeDB> db.students.find().pretty()
[
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05397'),
    rollNo: '23CM001',
    name: 'Ravi Kumar',
    branch: 'CSE-AIHL',
    year: 3,
    marks: 90,
    email: 'ravi@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05398'),
    rollNo: '23CM002',
    name: 'Anita Sharma',
    branch: 'CSE',
    year: 2,
    marks: 92,
    email: 'anita@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb05399'),
    rollNo: '23CM003',
    name: 'Kiran Reddy',
    branch: 'ECE',
    year: 3,
    marks: 68,
    email: 'kiran.reddy@example.com'
  },
  {
    _id: ObjectId('6ac0e1057d8e0c136eb0539a'),
    rollNo: '23CM004',
    name: 'Priya Singh',
    branch: 'CSE-AIHL',
    year: 4,
    marks: 45,
    email: 'priya@example.com'
  }
]
```

How to Run the MongoDB Assignment in VS Code

1. Install MongoDB Server and MongoDB Shell if they are not already installed.
2. Open Visual Studio Code and open the integrated terminal.
3. Check the shell using `mongosh --version`.
4. Check the MongoDB service using `Get-Service MongoDB`.
5. Open the shell using `mongosh`.
6. Enter the MongoDB commands from this assignment one by one.
7. Take screenshots of the commands and their outputs and paste them into the provided spaces.

Note: If the students collection already exists from an earlier attempt, `createCollection` may report that it already exists. In that case, continue with the required operations rather than creating a duplicate collection.

Conclusion

The MongoDB student information management system was implemented to store, retrieve, update, search, sort, and delete student records. Branch, marks, year, and rollNo queries were demonstrated. An index was created on rollNo and explain() was used to inspect index usage. The real-time extension queries were also performed.